PROJECT REPORT

AWS Multi-Region Disaster Recovery Using Terraform

Mumbai (ap-south-1) primary | Singapore (ap-southeast-1) recovery

Infrastructure as Code with reusable Terraform modules

Terraform | Amazon VPC | EC2 + ALB | S3 CRR | CloudWatch | EventBridge | SSM Automation

PREPARED BY

Parikshit

DATE

November 2026

TECHNOLOGY STACK

Terraform | AWS Provider 6.x | VS Code | Git and GitHub

Abstract

This project designs and implements a multi-region Disaster Recovery (DR) environment on Amazon Web Services using Terraform. Mumbai (**ap-south-1**) hosts the primary workload and Singapore (**ap-southeast-1**) hosts an independent recovery environment. Each region has its own VPC, public and private subnets, security groups, an EC2 application instance in a private subnet, and an internet-facing Application Load Balancer.

Object data is protected with Amazon S3 Cross-Region Replication between a primary and a secondary bucket. Infrastructure health is watched by Amazon CloudWatch alarms, which raise events that Amazon EventBridge routes to an AWS Systems Manager Automation runbook. The runbook marks the Singapore instance as *Failover-Ready*. An operator then verifies the recovery environment and manually switches DNS to the Singapore load balancer.

Everything is provisioned from reusable Terraform modules driven by two aliased AWS providers. A `terraform plan` run against the deployed stack reported no differences, and console screenshots confirm the VPCs, instances and load balancers in both regions. The report documents the design, the implementation, the failover procedure, the RTO and RPO model, and the current limitations together with a roadmap for fuller automation.

Project at a Glance

Primary region	Mumbai, ap-south-1 (production workload)
DR region	Singapore, ap-southeast-1 (recovery environment)
Infrastructure as Code	Terraform with seven reusable modules and two aliased AWS providers
Network	Two isolated VPCs: 10.1.0.0/16 and 10.2.0.0/16, public and private subnets in two AZs each
Compute and traffic	EC2 t3.micro (private subnet, no public IP) behind an internet-facing ALB, per region
Data protection	S3 versioning, AES256 encryption and Cross-Region Replication (Mumbai to Singapore)
Detection and automation	CloudWatch alarms, EventBridge rule, Systems Manager Automation runbook
Failover model	Event-triggered readiness tagging followed by operator-verified manual DNS switch

Table of Contents

Abstract 2

Project at a Glance . . . 2

1. Project Overview 5

1.1 Introduction . . . 5
1.2 Problem Statement . . . 5
1.3 Project Objectives . . . 5
1.4 Technologies and Services Used . . . 6
1.5 AWS Region Configuration . . . 6
1.6 Project Scope . . . 6
1.7 Expected Outcome . . . 7

2. Architecture Design 8

2.1 Architecture Overview . . . 8
2.2 High-Level Architecture . . . 8
2.3 Primary Region: Mumbai (ap-south-1) . . . 9
2.4 Secondary Region: Singapore (ap-southeast-1) . . . 9
2.5 Network Architecture . . . 10
2.6 Application Traffic Flow . . . 10
2.7 Data Replication Architecture . . . 10
2.8 Monitoring and Automation Architecture . . . 10
2.9 Disaster Recovery Traffic Flow . . . 11
2.10 Key Design Decisions . . . 11

3. AWS Services Explanation 12

3.1 Networking Services . . . 12
3.2 Amazon EC2 . . . 12
3.3 Application Load Balancer . . . 12
3.4 Amazon S3 and Cross-Region Replication . . . 13
3.5 AWS IAM . . . 13
3.6 Amazon CloudWatch . . . 13
3.7 Amazon EventBridge . . . 13
3.8 AWS Systems Manager Automation . . . 14
3.9 Terraform . . . 14

4. Terraform Implementation 15

4.1 Repository Structure . . . 15
4.2 Provider Configuration . . . 15
4.3 Root Module Composition . . . 15

4.4 Module Responsibilities . . . 16
4.5 EC2 Module Detail . . . 16
4.6 Deployment Workflow . . . 17
4.7 State Management . . . 17

5. Deployment Verification and Results 18

5.1 Terraform Configuration . . . 18
5.2 Configuration Matches Deployed Infrastructure . . . 19
5.3 Primary Region: Mumbai . . . 19
5.4 Secondary Region: Singapore . . . 21
5.5 Teardown with Terraform . . . 22

6. Disaster Recovery Operations 23

6.1 Automated Readiness Phase . . . 23
6.2 Operator Failover Runbook . . . 23
6.3 Failback . . . 23

7. Testing, RTO and RPO 24

7.1 Definitions . . . 24
7.2 RTO Model . . . 24
7.3 RPO Model . . . 24
7.4 Test Plan and Status . . . 25
7.5 DR Drill Measurement Record . . . 25

8. Security Considerations 26

9. Limitations and Future Enhancements 27

9.1 Current Limitations . . . 27
9.2 Future Enhancements . . . 27

10. Conclusion 28

Appendix A: Resource Inventory 29

1. Project Overview

1.1 Introduction

This project builds a multi-region Disaster Recovery infrastructure on AWS using Terraform as the Infrastructure as Code (IaC) tool. Two regions are used: Mumbai (**ap-south-1**) as the primary region and Singapore (**ap-southeast-1**) as the secondary, disaster recovery region.

The primary environment hosts the application. The secondary environment provides an alternative infrastructure that can be used to restore application availability during a regional outage. The project integrates AWS networking, compute, load balancing, storage replication, monitoring and event-driven automation to demonstrate a practical recovery strategy.

1.2 Problem Statement

Applications deployed in a single cloud region can become unavailable because of regional service disruptions, infrastructure failures or unexpected outages. Such incidents affect availability and can cause data loss when backup and recovery mechanisms are missing.

This project addresses these risks by establishing a second AWS environment in a different geography, replicating object data across regions, and using monitoring and automation to support recovery operations.

1.3 Project Objectives

1. Design a multi-region AWS architecture with separate primary and secondary environments.
2. Provision cloud infrastructure using reusable Terraform modules.
3. Configure VPC networking, public subnets, private subnets and security groups.
4. Deploy EC2 instances to host application workloads in both regions.
5. Configure Application Load Balancers to distribute application traffic.
6. Implement Amazon S3 Cross-Region Replication to maintain a secondary copy of stored objects.
7. Configure Amazon CloudWatch alarms to monitor infrastructure health.
8. Integrate Amazon EventBridge with AWS Systems Manager Automation for event-driven DR readiness.
9. Implement a manual DNS switching strategy to redirect traffic to the secondary region after verification.
10. Test the recovery environment and document disaster recovery performance.

1.4 Technologies and Services Used

Category	Technologies
Cloud platform	Amazon Web Services (AWS)
Infrastructure as Code	Terraform, Terraform modules, AWS provider (v6.71.0 locked in the project)
Networking	Amazon VPC, public and private subnets, Internet Gateway, route tables, security groups
Compute and load balancing	Amazon EC2, Application Load Balancer, target groups
Storage and data protection	Amazon S3, versioning, Cross-Region Replication, server-side encryption
Monitoring and automation	Amazon CloudWatch and alarms, Amazon EventBridge, AWS Systems Manager Automation
Identity and access	AWS IAM roles, policies and service permissions
Development environment	Windows 11, PowerShell, Visual Studio Code, Git and GitHub

1.5 AWS Region Configuration

Configuration	Primary region	Secondary region
AWS region	Mumbai	Singapore
Region code	ap-south-1	ap-southeast-1
Purpose	Production environment	Disaster recovery environment
VPC CIDR	10.1.0.0/16	10.2.0.0/16
EC2 instance	t3.micro	t3.micro
Load balancer	Application Load Balancer	Application Load Balancer
S3 bucket	Primary storage	Replicated storage

1.6 Project Scope

The project covers multi-region infrastructure deployment, provisioning through Terraform, network isolation and security, application hosting on EC2, traffic distribution through Application Load Balancers, cross-region object replication, monitoring through CloudWatch, event-driven automation using EventBridge and Systems Manager, manual traffic redirection to the secondary region, and DR testing with performance measurement.

Scope boundary
The current implementation uses an **event-triggered readiness workflow with manual DNS failover**. It does not implement fully automated application failover. This is a deliberate, controlled design: an operator verifies the recovery environment before users are redirected.

1.7 Expected Outcome

The expected outcome is a reusable multi-region DR infrastructure that shows how AWS services work together to improve application resilience. If the Mumbai environment becomes unavailable, the Singapore environment can be verified and prepared for recovery. Replicated S3 objects provide an additional copy of stored data, and the secondary Application Load Balancer provides an alternative application entry point.

2. Architecture Design

2.1 Architecture Overview

The infrastructure is distributed across two geographically separate AWS regions. Mumbai is the primary application environment; Singapore maintains a secondary environment for recovery. Both regions contain independent networking, compute and load-balancing resources, and S3 Cross-Region Replication maintains replicated object data between them. Monitoring and DR readiness are managed through CloudWatch, EventBridge and Systems Manager Automation. Traffic is redirected to Singapore through manual DNS switching once the recovery environment has been verified.

2.2 High-Level Architecture

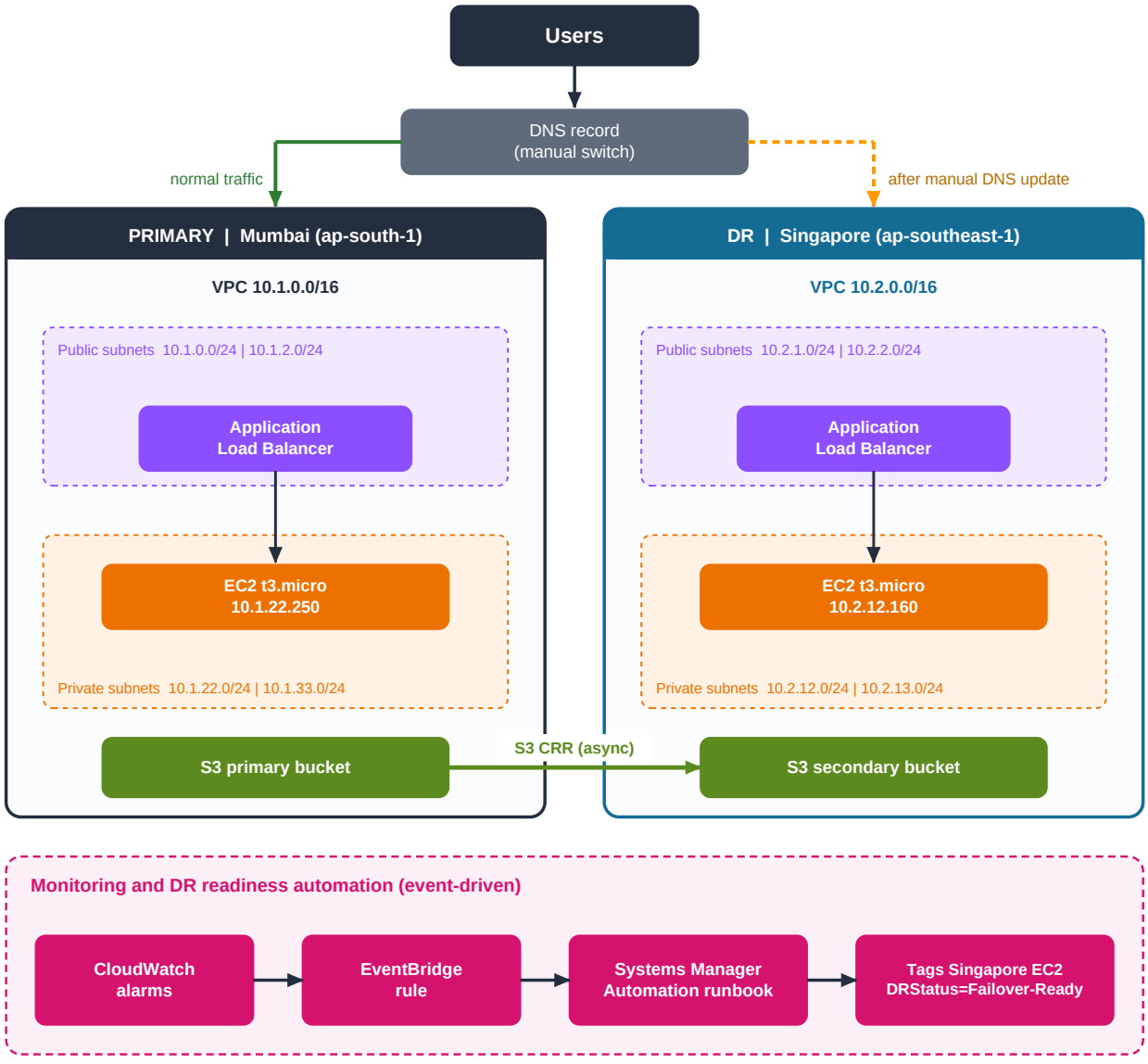


Figure 2.1: Multi-region DR architecture (green: normal traffic path, dashed orange: failover path after manual DNS update)

Note
The diagram shows the intended recovery architecture. The current SSM runbook performs a tagging operation on the secondary EC2 instance. Starting the instance, application validation and the DNS change remain manual.

2.3 Primary Region: Mumbai (ap-south-1)

The primary region hosts the main application infrastructure and serves normal traffic.

Resource	Configuration
VPC	10.1.0.0/16
Public subnets	10.1.0.0/24 and 10.1.2.0/24
Private subnets	10.1.22.0/24 and 10.1.33.0/24
EC2 instance	t3.micro, private IP 10.1.22.250
Application Load Balancer	Primary ALB (internet-facing)
S3 bucket	parikshitarer-dr-primary-unique123

Normal workflow

1. Users reach the application through the configured DNS endpoint, which resolves to the primary ALB.
2. The ALB receives HTTP requests and forwards them to the registered EC2 target.
3. The EC2 instance processes the request; application data is stored in the primary S3 bucket where applicable.
4. CloudWatch monitors the EC2 instance.

2.4 Secondary Region: Singapore (ap-southeast-1)

The secondary region provides an alternative environment that remains available for recovery operations.

Resource	Configuration
VPC	10.2.0.0/16
Public subnets	10.2.1.0/24 and 10.2.2.0/24
Private subnets	10.2.12.0/24 and 10.2.13.0/24
EC2 instance	t3.micro, private IP 10.2.12.160
Application Load Balancer	Secondary ALB (internet-facing)
S3 bucket	parikshitarer-dr-secondary-unique123

Recovery workflow

1. The secondary ALB provides an alternative application endpoint and the secondary EC2 instance hosts the recovery workload.
2. Replicated S3 objects are available in the secondary bucket.
3. During a disaster the recovery environment is verified by an operator.

4. DNS is manually updated to direct users to the secondary ALB.

2.5 Network Architecture

Each region has an independent VPC. The CIDR ranges differ so that private address spaces never overlap, which keeps future connectivity options such as peering or VPN open.

- **Public subnets** hold the internet-facing Application Load Balancers and are associated with public route tables that route 0.0.0.0/0 to the Internet Gateway.
- **Private subnets** hold the EC2 application instances. Instances receive no public IP address and are reachable only through the ALB security group.

Security group design

Component	Allowed traffic
ALB security group	HTTP, TCP 80, from 0.0.0.0/0
EC2 security group	HTTP, TCP 80, from the ALB security group only
EC2 public IP	Disabled

This prevents direct public HTTP access to the instances. The architecture does not include NAT gateways, VPC peering or a site-to-site VPN. The two regions operate independently and S3 replication is the only cross-region data path.

2.6 Application Traffic Flow

During normal operation a request follows this path: **User** to **DNS resolution** to the **primary ALB** to the **primary target group** to the **primary EC2 instance**, and the response returns along the same path. The target group performs an HTTP health check on the `/` path and expects HTTP 200.

2.7 Data Replication Architecture

Amazon S3 Cross-Region Replication (CRR) keeps a secondary copy of stored objects. Objects written to the primary bucket in Mumbai are replicated asynchronously to the secondary bucket in Singapore.

Replication components: versioning enabled on both buckets, AES256 server-side encryption, a dedicated S3 replication IAM role, an IAM replication policy, and an enabled replication rule.

> **Consistency note**
> Replication is asynchronous, so the latest object is not guaranteed to be present in the secondary bucket immediately. Objects that existed before replication was enabled need S3 Batch Replication to be copied.

2.8 Monitoring and Automation Architecture

CloudWatch monitors EC2 metrics in both regions. EventBridge receives matching alarm-state events and invokes the Systems Manager Automation workflow. The automation document is named `Parikshit-disaster-recovery-dr-runbook` and tags the Singapore EC2 instance with:

- `DRStatus = Failover-Ready`
- `DRActivationSource = Automated`

The tags identify the instance as marked for DR readiness. The runbook does not start the instance, validate the application or change DNS records.

2.9 Disaster Recovery Traffic Flow

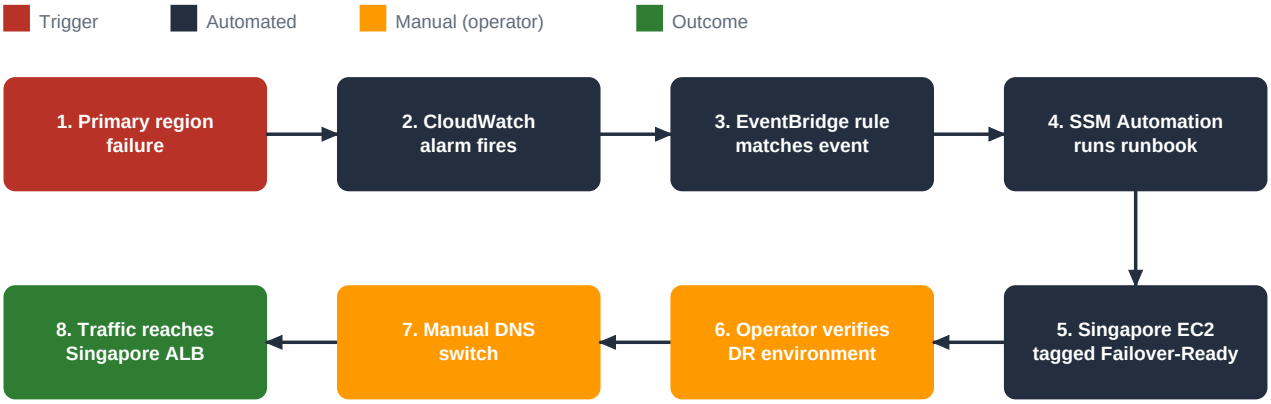


Figure 2.2: Failover sequence from primary failure to traffic in Singapore

The operator must confirm that the secondary application is operational before changing DNS. This avoids redirecting users to an environment that is not ready, and it prevents an unnecessary switch when an alarm is a false positive.

2.10 Key Design Decisions

Decision	Rationale and trade-off
Non-overlapping VPC CIDRs	Keeps the address plan clean and leaves room for peering or VPN later.
EC2 in private subnets behind an ALB	Removes direct public exposure of servers. Instances have no outbound internet path because no NAT is deployed.
Independent regional stacks	A failure in one region cannot cascade to the other. The cost is that data and configuration must be kept in step.
Manual DNS switch	An operator verifies the DR environment first, avoiding premature or false-positive failover. The cost is a longer RTO.
S3 Cross-Region Replication	Low-effort, managed protection for object data with versioning. Replication is asynchronous, which shapes the RPO.
Modular Terraform	Primary and secondary stacks reuse the same modules, giving consistent and repeatable provisioning.

3. AWS Services Explanation

This chapter explains each AWS service used in the project, how it is configured, and the role it plays in the DR design.

3.1 Networking Services

Service	Purpose	Role in this project
Amazon VPC	Isolated virtual network inside AWS.	Two VPCs (10.1.0.0/16 and 10.2.0.0/16) give each region an independent network with no IP overlap.
Subnets	Network segments inside a VPC.	Public subnets hold the ALBs; private subnets hold the EC2 instances.
Internet Gateway	Connects a VPC to the internet.	Provides internet connectivity for the public subnets that contain the ALBs.
Route tables	Rules that direct network traffic.	Public route tables hold a local route plus 0.0.0.0/0 to the Internet Gateway.
Security groups	Stateful virtual firewalls.	ALB accepts HTTP from the internet; EC2 accepts HTTP only from the ALB security group.

3.2 Amazon EC2

Amazon Elastic Compute Cloud provides the virtual servers that host the application workload in each region.

Instance type	t3.micro
Root volume	8 GB gp3, encrypted
Public IP	Disabled (private subnet)
AMI	Latest Canonical Ubuntu 24.04 (Noble) amd64 server image, resolved with a Terraform data source
Placement	Mumbai primary instance and Singapore recovery instance

3.3 Application Load Balancer

The Application Load Balancer distributes incoming HTTP traffic across healthy registered targets. Each region has its own ALB, and the secondary ALB is the alternative entry point during recovery.

Type and scheme	Application, internet-facing, IPv4, two Availability Zones
Listener	HTTP, port 80
Target group	HTTP, port 80
Health check	Path `/`, expects HTTP 200

3.4 Amazon S3 and Cross-Region Replication

Amazon S3 stores application objects. Both buckets have versioning enabled, AES256 server-side encryption and public access blocked. CRR automatically replicates eligible objects from the primary bucket to the secondary bucket in another region.

1. Enable versioning on both buckets.
2. Create an IAM replication role.
3. Attach the replication policy.
4. Create an enabled replication rule on the source bucket.
5. Upload an eligible object; S3 replicates it to the destination bucket.

Because replication is asynchronous, replication delay has to be considered when calculating the RPO (see Chapter 7).

3.5 AWS IAM

IAM manages authentication and authorization. The project uses service-specific roles and policies instead of broad permissions, granting access to:

- **Amazon S3** for cross-region replication.
- **Amazon EventBridge** for starting the Systems Manager Automation.
- **AWS Systems Manager** for executing the automation actions.

3.6 Amazon CloudWatch

CloudWatch collects metrics, logs and events. Two alarms monitor each EC2 instance:

Alarm	Metric	Condition	Purpose
High CPU	CPUUtilization	Greater than 80%	Detects overload that may affect availability.
Status check	StatusCheckFailed	Any failed status check	Detects instance or system level failure.

When an alarm condition is met the alarm enters the ALARM state, which generates an event for EventBridge.

3.7 Amazon EventBridge

EventBridge is an event-driven service that routes events to targets. In this project it matches CloudWatch alarm-state change events and invokes the Systems Manager Automation workflow:

1. A CloudWatch alarm changes state.
2. An event is generated.
3. EventBridge evaluates its rule.
4. A matching event invokes the configured target.
5. Systems Manager Automation executes the runbook.

3.8 AWS Systems Manager Automation

Systems Manager Automation automates operational tasks on AWS resources. The runbook `Parikshit-disaster-recovery-dr-runbook` applies the tags `DRStatus=Failover-Ready` and `DRActivationSource=Automated` to the Singapore EC2 instance, demonstrating event-driven DR readiness. It does not start the instance or perform a complete application failover.

3.9 Terraform

Terraform provisions all of the following resources: VPCs, subnets, Internet Gateways, route tables, security groups, EC2 instances, Application Load Balancers, target groups, S3 buckets, replication configuration, IAM roles and policies, CloudWatch alarms, EventBridge rules and Systems Manager documents. Its benefits for this project are repeatable deployment, version-controlled configuration, reusable modules, consistent provisioning and easier management.

4. Terraform Implementation

4.1 Repository Structure

The root module composes seven child modules. Each module has its own `main.tf`, `variables.tf`, outputs and `versions.tf`.

```
AWS-Disaster-Recovery-via-IAC/
|-- main.tf                root module: provider-aliased module calls
|-- provider.tf            aws.primary (Mumbai) and aws.secondary (Singapore)
|-- variables.tf
|-- outputs.tf
|-- versions.tf
|-- terraform.tfvars
|-- .terraform.lock.hcl    provider version lock
|
`-- modules/
    |-- networking/        VPC, subnets, Internet Gateway, route tables
    |-- security/          ALB and EC2 security groups
    |-- ec2/               application instance
    |-- alb/               load balancer, listener, target group
    |-- monitoring/        CloudWatch alarms
    |-- s3-replication/    buckets, versioning, encryption, CRR, IAM
    `-- dr-automation/     SSM runbook, EventBridge rule, IAM roles
```

4.2 Provider Configuration

Two aliased AWS providers let a single configuration manage both regions: `aws.primary` targets ap-south-1 and `aws.secondary` targets ap-southeast-1. Each module call receives the provider it should use through a `providers` block. The project is locked to the HashiCorp AWS provider version 6.71.0.

4.3 Root Module Composition

The root `main.tf` instantiates the same modules once per region and names resources with a region suffix, for example `${var.project_name}-primary` and `${var.project_name}-secondary`. Networking is called twice with its own CIDR blocks, public and private subnet lists and availability zones.

```
module "primary_networking" {
  source    = "./modules/networking"
  providers = { aws = aws.primary }

  project_name         = "${var.project_name}-primary"
  vpc_cidr             = var.primary_vpc_cidr
  public_subnet_cidrs  = var.primary_public_subnet_cidrs
  private_subnet_cidrs = var.primary_private_subnet_cidrs
  availability_zones   = var.primary_availability_zones
  environment          = var.environment
}

module "secondary_networking" {
  source    = "./modules/networking"
  providers = { aws = aws.secondary }
  # same inputs, secondary_* variables
}
```

Excerpt reproduced from the project's root main.tf (Figure 5.2); the secondary block uses the same input names with secondary_* variables.

4.4 Module Responsibilities

Module	Responsibility
networking	VPC, public and private subnets across two AZs, Internet Gateway, route tables and associations.
security	Security groups: ALB (HTTP from internet) and EC2 (HTTP from the ALB security group).
ec2	Application instance with an AMI looked up at plan time, no public IP, encrypted gp3 root volume and an instance profile.
alb	Application Load Balancer, HTTP listener, target group with health check and target registration.
monitoring	CloudWatch alarms for CPU utilization and EC2 status checks.
s3-replication	Primary and secondary buckets, versioning, AES256 encryption, public access block, replication IAM role, policy and rule.
dr-automation	Systems Manager runbook (aws_ssm_document), EventBridge rule and target, and IAM roles and policy attachments.

4.5 EC2 Module Detail

The EC2 module resolves the image dynamically instead of hard-coding an AMI ID. It selects the most recent Canonical-owned Ubuntu 24.04 (Noble) amd64 server image and launches the instance in the supplied private subnet with the supplied security group, no public IP and an IAM instance profile.

```
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]           # Canonical

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
  }
}

resource "aws_instance" "ec2_instance" {
  ami                         = data.aws_ami.ubuntu.id
  instance_type               = var.instance_type
  subnet_id                   = var.subnet_id
  vpc_security_group_ids      = [var.security_group_id]
  associate_public_ip_address = false
  iam_instance_profile        = var.iam_instance_profile
  ...
}
```

4.6 Deployment Workflow

Command	Purpose
terraform init	Download providers and initialise modules.
terraform fmt -recursive	Format all configuration files.
terraform validate	Check configuration syntax and consistency.
terraform plan -out=tfplan	Preview changes and save the plan for review.
terraform apply tfplan	Create or update the infrastructure in both regions.
terraform destroy	Remove all resources after testing to stop charges.

4.7 State Management

State is currently stored locally (`terraform.tfstate` with its backup file). This is adequate for a single-developer project. For team use, a remote backend such as an S3 bucket with state locking is recommended (see Chapter 9).

5. Deployment Verification and Results

This chapter presents evidence that the Terraform configuration was applied successfully and that the infrastructure exists in both regions.

5.1 Terraform Configuration

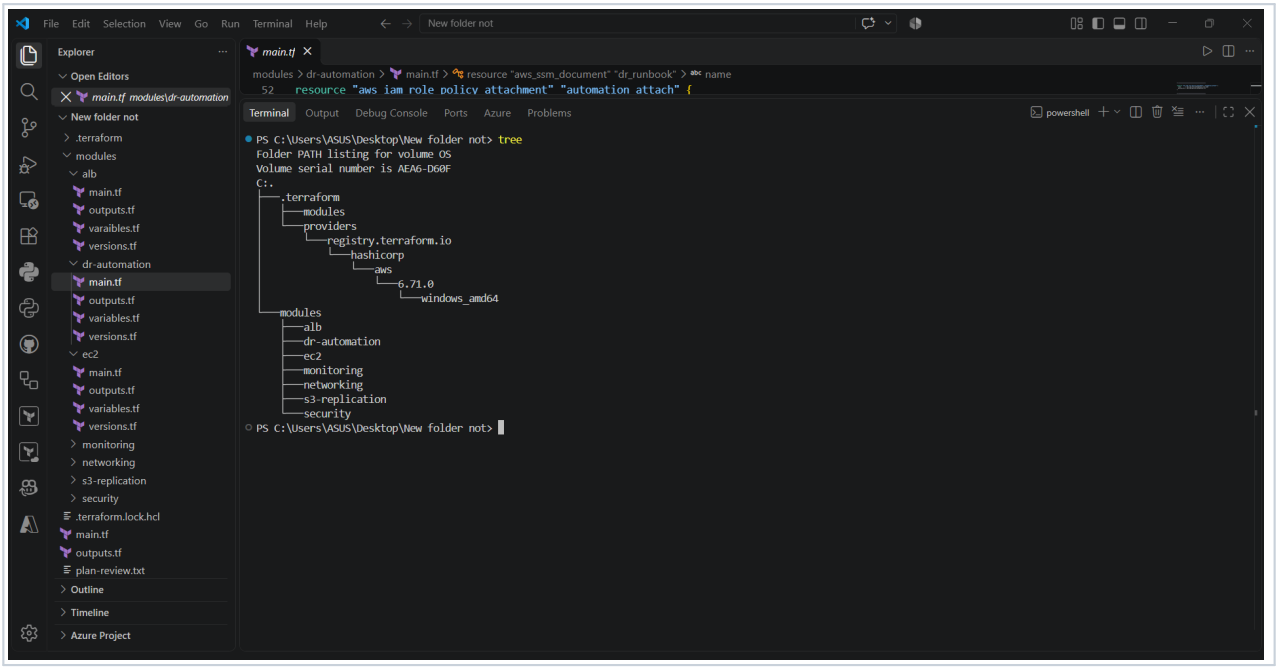


Figure 5.1: Project in Visual Studio Code with the seven module directories, root files and the Terraform provider tree (AWS provider 6.71.0).

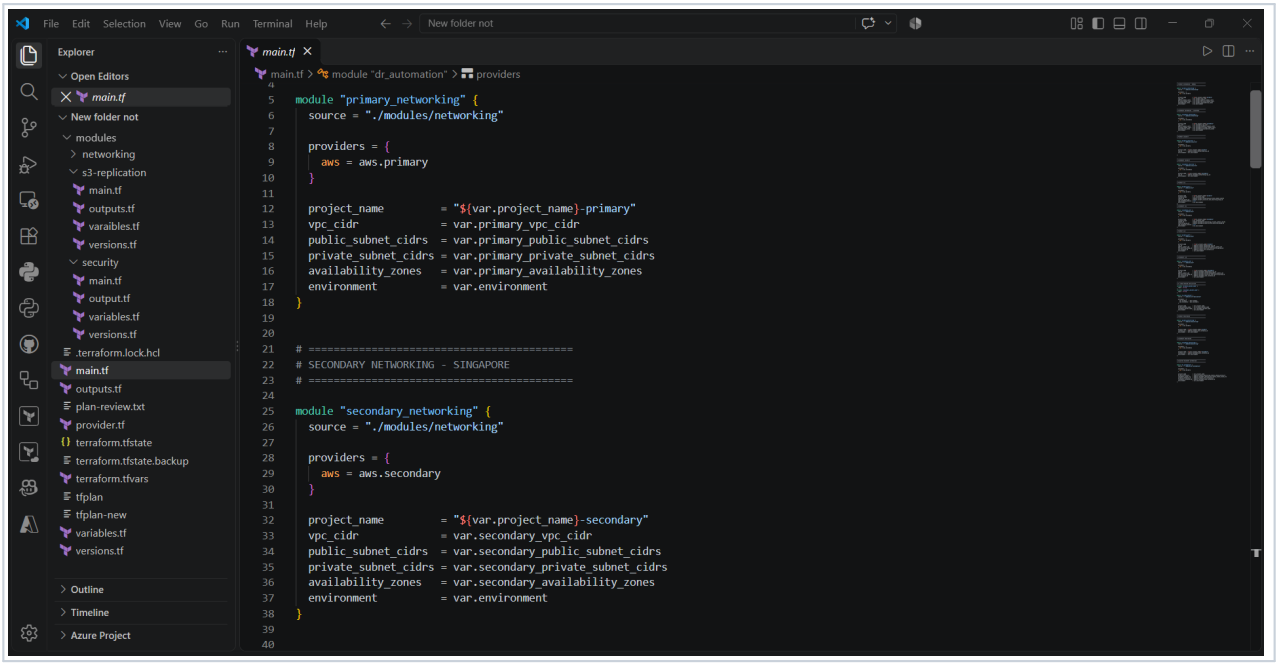


Figure 5.2: Root main.tf calling the networking module once per region with aws.primary and aws.secondary providers.

5.2 Configuration Matches Deployed Infrastructure

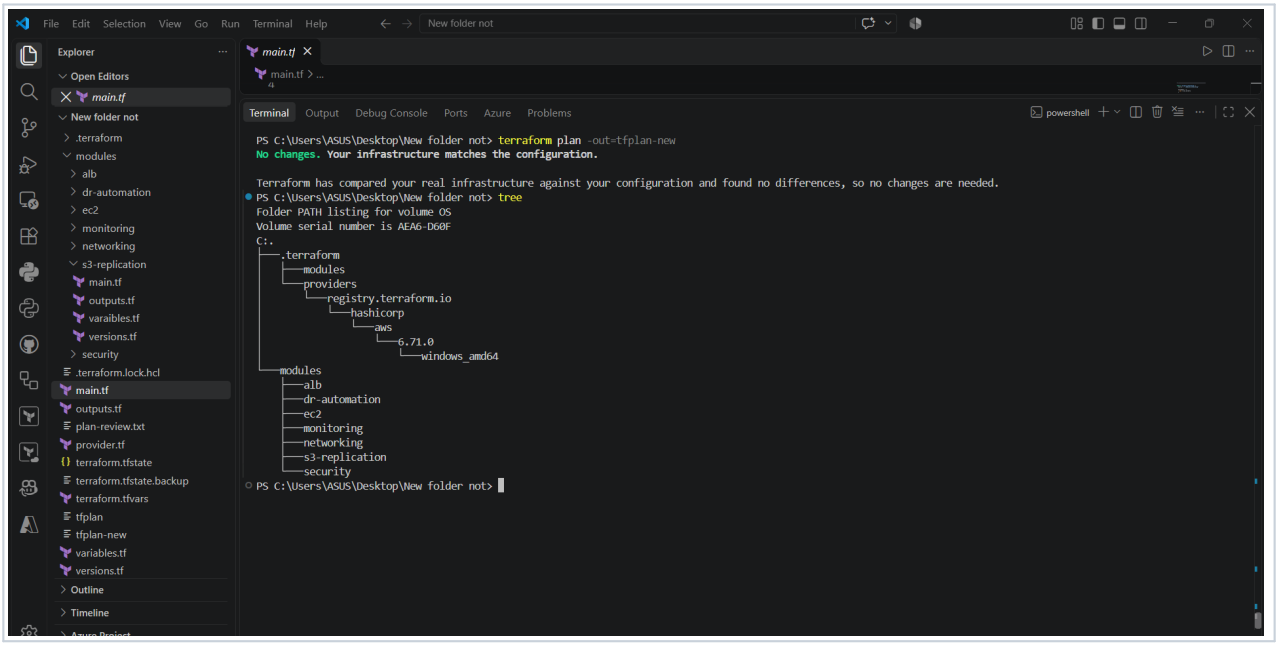


Figure 5.3: terraform plan reports no changes: the real infrastructure matches the configuration.

The plan output confirms there is no configuration drift between the Terraform code and the resources running in AWS.

5.3 Primary Region: Mumbai

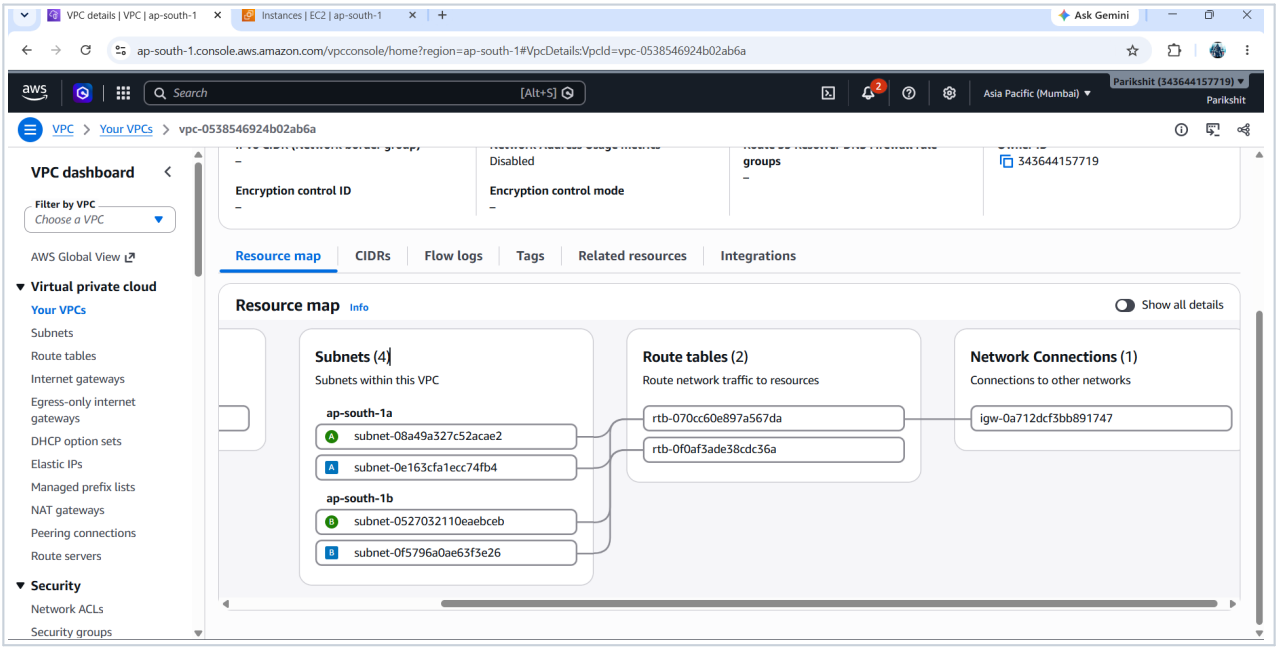


Figure 5.4: Mumbai VPC resource map showing four subnets across ap-south-1a and ap-south-1b, two route tables and an Internet Gateway.

The Mumbai VPC (vpc-0538546924b02ab6a) contains four subnets split evenly across two Availability Zones and two route tables. Only one route table is connected to the Internet Gateway, which matches the public and private subnet design.

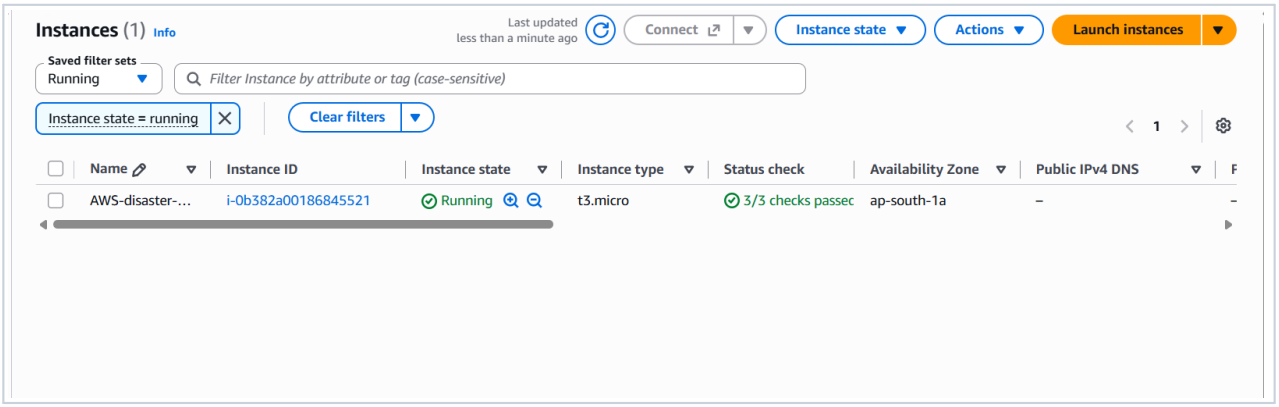


Figure 5.5: Mumbai EC2 instance running as t3.micro in ap-south-1a with 3/3 status checks passed and no public IPv4 DNS.

The instance is healthy (3/3 checks) and shows no public IPv4 DNS name, confirming that it is reachable only through the load balancer.

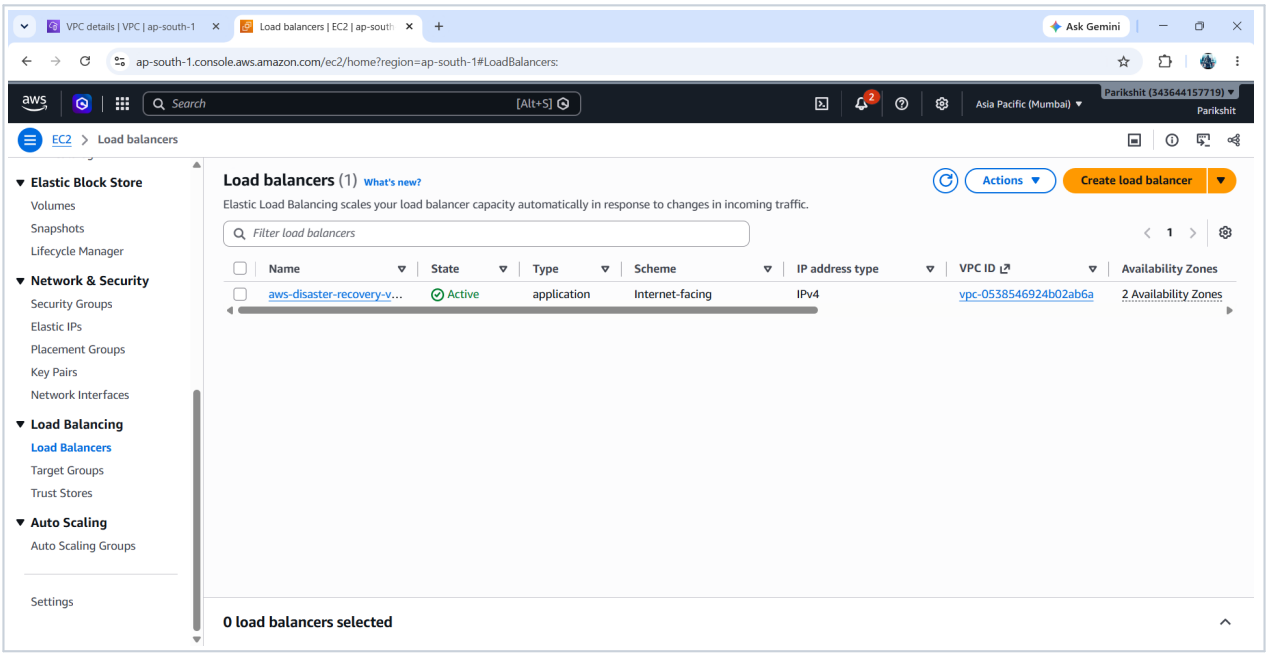


Figure 5.6: Mumbai Application Load Balancer, Active, internet-facing, in the Mumbai VPC across two Availability Zones.

5.4 Secondary Region: Singapore

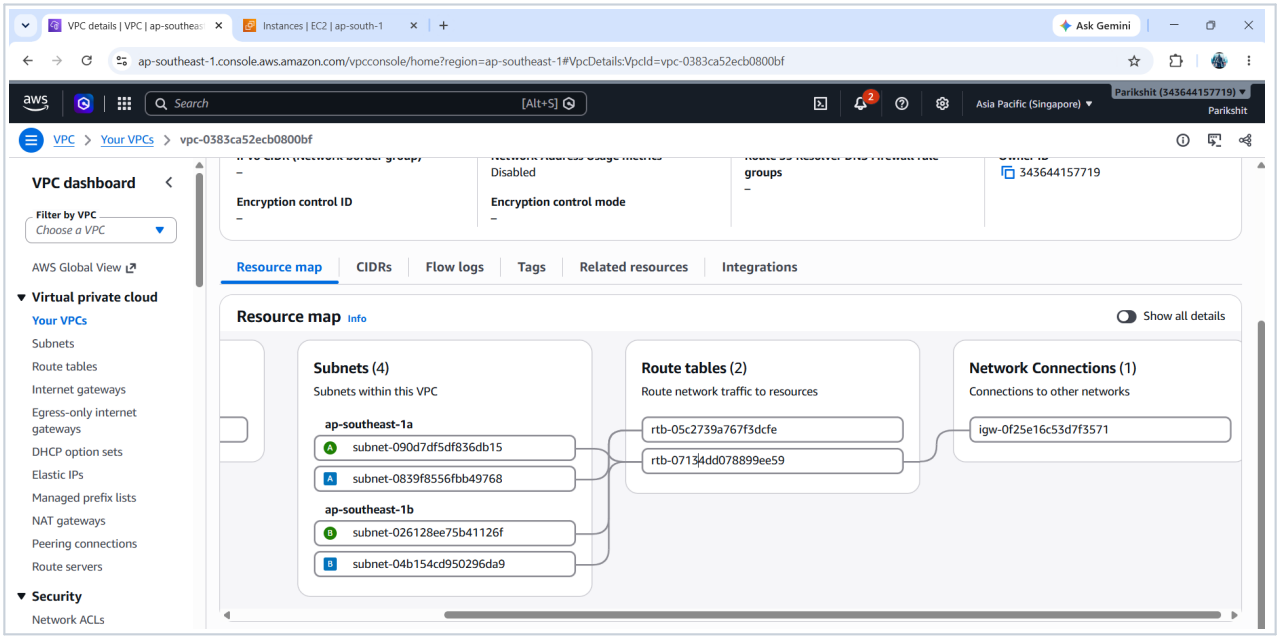


Figure 5.7: Singapore VPC resource map showing four subnets across ap-southeast-1a and ap-southeast-1b, two route tables and an Internet Gateway.

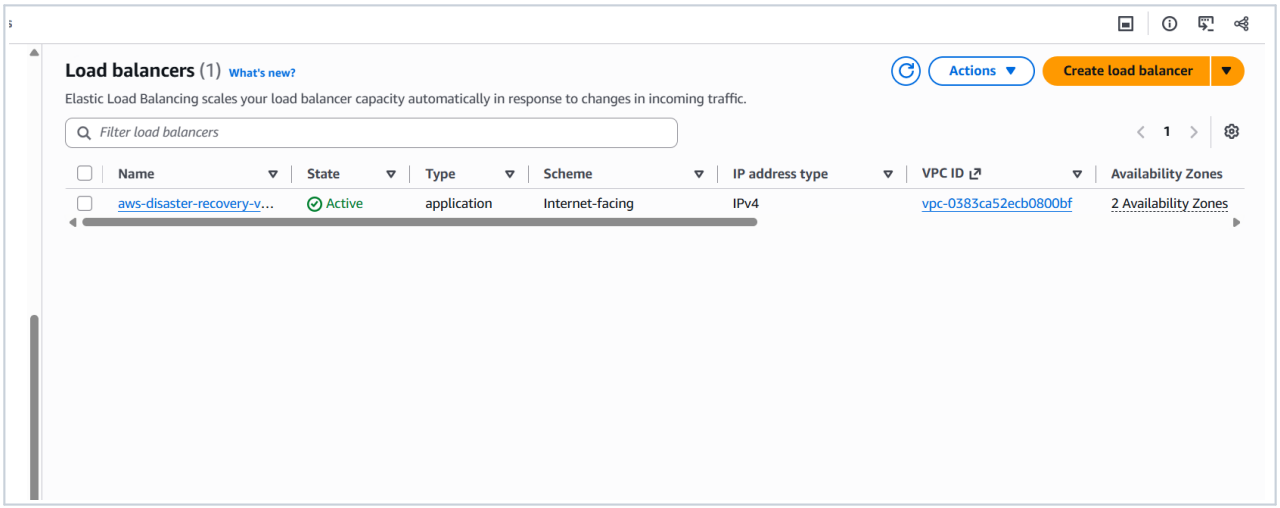


Figure 5.8: Singapore Application Load Balancer, Active, internet-facing, in the Singapore VPC across two Availability Zones.

The Singapore VPC (vpc-0383ca52ecb0800bf) mirrors the Mumbai design, and its load balancer is in the Active state, ready to serve as the recovery entry point.

5.5 Teardown with Terraform

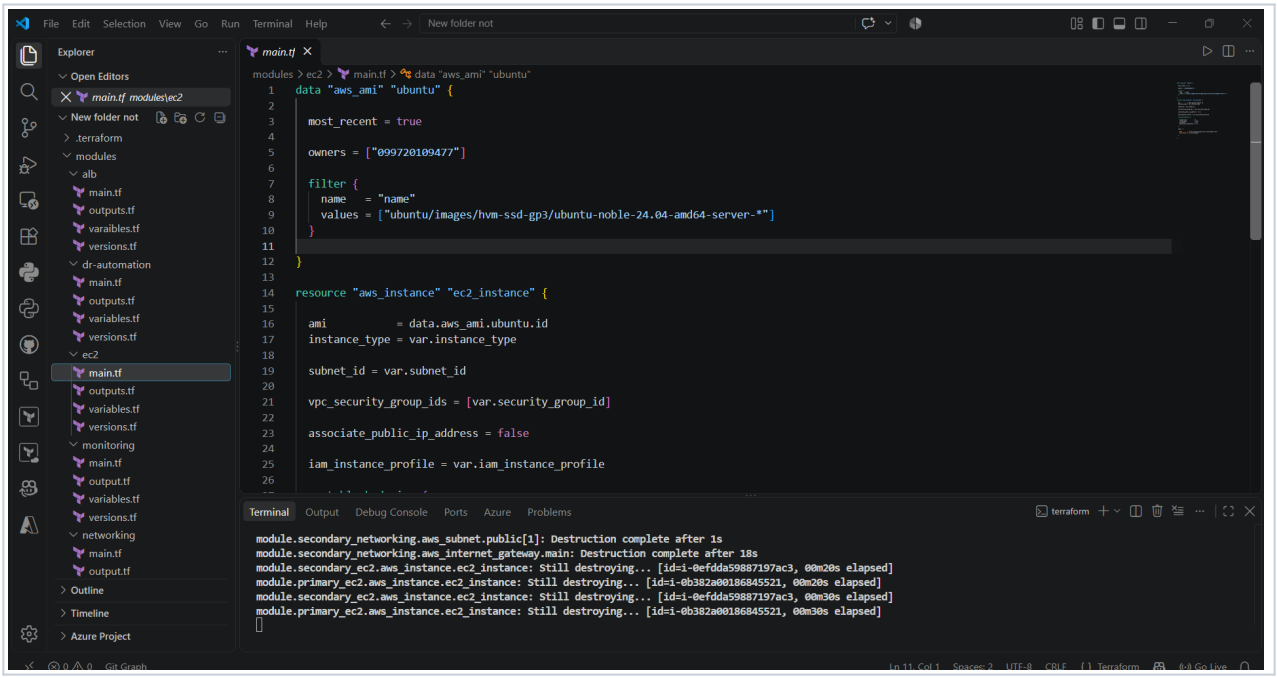


Figure 5.9: EC2 module source (top) and terraform destroy removing the primary and secondary resources (bottom).

After verification the environment was removed with `terraform destroy`. Because the whole stack is code, it can be rebuilt in both regions on demand, which also demonstrates the recoverability of the infrastructure itself.

6. Disaster Recovery Operations

6.1 Automated Readiness Phase

When a monitored CloudWatch alarm enters the ALARM state, EventBridge matches the event and starts the Systems Manager Automation runbook. The runbook tags the Singapore instance `DRStatus=Failover-Ready` and `DRActivationSource=Automated`, giving the operator a clear signal and an audit trail that the readiness workflow ran.

6.2 Operator Failover Runbook

After the automated phase an operator completes the failover. The steps below are the recommended procedure for this architecture.

1. **Confirm the incident.** Check the primary ALB and EC2 status and confirm that the alarm reflects a real outage and not a transient spike.
2. **Check the readiness tags** on the Singapore instance.
3. **Verify the secondary target health** so that the Singapore ALB has a healthy target.
4. **Validate the application** by requesting the Singapore ALB DNS name directly and checking for HTTP 200.
5. **Verify replicated data** in the secondary S3 bucket.
6. **Switch DNS** so that the application record points to the Singapore ALB DNS name.
7. **Validate end to end** through the public hostname and monitor the Singapore environment.
8. **Record timestamps** for the incident, decision and DNS change to calculate the achieved RTO.

Example verification commands

```
# Readiness tags on the Singapore instance
aws ec2 describe-tags --region ap-southeast-1 \
    --filters "Name=resource-id,Values=<singapore-instance-id>"

# Target health behind the Singapore ALB
aws elbv2 describe-target-health --region ap-southeast-1 \
    --target-group-arn <secondary-target-group-arn>

# Application check through the Singapore ALB
curl -I http://<secondary-alb-dns-name>/

# Replicated objects in the secondary bucket
aws s3 ls s3://parikshitarer-dr-secondary-unique123 --region ap-southeast-1
```

Preparation tip
Lower the DNS record TTL ahead of time (for example to 60 seconds). A long TTL is the single largest uncontrolled contributor to recovery time after the DNS switch.

6.3 Failback

When the Mumbai region is healthy again: verify the primary application and target health, reconcile any objects written to the Singapore bucket during the outage (S3 replication in this design flows from Mumbai to Singapore), then switch DNS back to the primary ALB and monitor. Clear or reset the DR tags on the Singapore instance.

7. Testing, RTO and RPO

7.1 Definitions

- **RTO (Recovery Time Objective)**: the time from the start of an outage until users are served from the recovery environment.
- **RPO (Recovery Point Objective)**: the maximum window of data that can be lost, measured back from the moment of failure.

7.2 RTO Model

The RTO for this design is the sum of the stages below. The manual stages dominate, which is the trade-off accepted in exchange for a verified failover.

Stage	Type	What determines its duration
Failure detection	Automated	Alarm period multiplied by evaluation periods. EC2 basic monitoring publishes metrics at 5-minute intervals.
Event routing and runbook	Automated	EventBridge delivery and SSM Automation execution, typically seconds.
Operator verification	Manual	Time to confirm the incident, check target health and validate the application.
DNS change	Manual	Time to edit the record.
DNS propagation	External	Record TTL and resolver caching.

7.3 RPO Model

The RPO for object data is governed by S3 replication latency. Replication is asynchronous, so objects uploaded shortly before the failure may not yet exist in the secondary bucket. Without S3 Replication Time Control there is no service-level guarantee on replication time; enabling it provides a 15-minute replication target for most objects. The EC2 instances hold no replicated state, so application state kept outside S3 is not covered by this RPO.

7.4 Test Plan and Status

Test	Method	Expected result	Status
Infrastructure deployment	terraform apply	Resources created in both regions	Verified (Figs 5.3 to 5.8)
Configuration drift	terraform plan	No changes	Verified (Fig 5.3)
Instance health	EC2 console	3/3 status checks passed, no public IP	Verified (Fig 5.5)
Load balancers active	EC2 console	ALBs in Active state in both regions	Verified (Figs 5.6, 5.8)
S3 replication	Upload an object to the primary bucket	Object appears in the secondary bucket	Record replication time
Alarm to runbook	Force an alarm (for example stop the instance)	EventBridge starts runbook; DR tags applied	Record result
Secondary application	curl the Singapore ALB	HTTP 200 from the recovery instance	Record result
DNS failover	Update DNS to the secondary ALB	Users served from Singapore	Record RTO

7.5 DR Drill Measurement Record

Complete this table during a failover drill to document the achieved RTO and RPO.

Measurement	Timestamp / value	Notes
Outage simulated (T0)		
Alarm entered ALARM state		
SSM runbook completed (tags applied)		
Operator verification completed		
DNS record updated		
First successful request via Singapore ALB		
Achieved RTO (T0 to first Singapore response)		
Last object replicated before T0		
Observed RPO (T0 minus last replicated write)		

8. Security Considerations

Area	Control in this project
Network exposure	EC2 instances sit in private subnets with no public IP. Only the ALBs are internet-facing.
Least-privilege traffic	EC2 security group accepts HTTP only from the ALB security group.
Data at rest	S3 buckets use AES256 server-side encryption with public access blocked; EC2 root volumes are encrypted.
Data protection	S3 versioning protects against accidental overwrite and deletion and is required for replication.
Identity	Service-specific IAM roles for S3 replication, EventBridge and Systems Manager, instead of broad permissions.
Change control	All infrastructure is declared in version-controlled Terraform and reviewed with terraform plan before apply.
Instance access	An IAM instance profile is attached to the EC2 instance for managed, key-less access patterns.

Recommended hardening: add HTTPS with ACM certificates on the ALBs, restrict the ALB ingress where possible, add VPC endpoints for private access to AWS services, and enable AWS CloudTrail and VPC flow logs for audit and investigation.

9. Limitations and Future Enhancements

9.1 Current Limitations

- Failover is **not fully automated**: the runbook only tags the Singapore instance; verification and the DNS change are manual.
- An EC2 alarm reflects the health of one instance, not a full regional outage, so detection is limited.
- S3 replication is asynchronous and protects objects only. There is no database or EBS-level replication.
- The HTTP-only listener does not encrypt traffic in transit.
- No NAT gateway or VPC endpoints are deployed, so private instances have no outbound internet or private AWS service path.
- Terraform state is local, which does not support team collaboration or locking.
- Running two ALBs and two instances incurs hourly cost, so the stack is best created for tests and destroyed afterwards.

9.2 Future Enhancements

Enhancement	Benefit
Route 53 failover routing with health checks	Automatic DNS failover and reduced RTO.
Extend the SSM runbook	Start the DR instance, validate the application and optionally update DNS automatically.
Pilot-light or warm-standby with Auto Scaling	Scale the DR environment up on demand and remove the single-instance dependency.
HTTPS with AWS Certificate Manager	Encrypt traffic in transit on both ALBs.
SNS notifications	Alert operators when the readiness workflow runs.
Alarms on ALB and target health	Detect application-level failure, not only instance-level failure.
Cross-region database replica or backups with AWS Backup	Protect stateful data beyond S3 objects.
S3 Replication Time Control and Batch Replication	Tighter, measurable RPO and coverage for pre-existing objects.
Remote Terraform state with locking	Safe team workflows and state durability.
CI/CD with GitHub Actions	Automated fmt, validate and plan on every pull request.

10. Conclusion

The project demonstrates a multi-region AWS Disaster Recovery solution built entirely with Infrastructure as Code. The Mumbai environment handles normal application traffic while the Singapore environment provides an independent recovery stack with its own network, compute and load balancer. S3 Cross-Region Replication protects object data, CloudWatch monitors instance health, and EventBridge connects alarm events to a Systems Manager Automation runbook that marks the recovery instance as ready.

A manual, operator-verified DNS switch keeps failover controlled and avoids redirecting users to an unverified environment. The Terraform plan reporting no changes, together with the console evidence for VPCs, instances and load balancers in both regions, confirms that the deployed infrastructure matches the code. Because the whole stack can be destroyed and recreated on demand, the infrastructure itself is repeatable and recoverable.

The architecture can be extended with automated instance recovery, application health validation, HTTPS, backup policies, Route 53 failover routing and richer orchestration to reduce RTO and broaden the protected data set.

References

- AWS Documentation: Amazon VPC User Guide.
- AWS Documentation: Elastic Load Balancing, Application Load Balancers.
- AWS Documentation: Amazon S3 User Guide, Replicating objects (Cross-Region Replication).
- AWS Documentation: Amazon CloudWatch alarms; Amazon EventBridge User Guide.
- AWS Documentation: AWS Systems Manager Automation.
- AWS Well-Architected Framework: Reliability Pillar, Disaster Recovery strategies.
- HashiCorp Terraform Documentation and Terraform AWS Provider Registry.

Appendix A: Resource Inventory

Resource identifiers observed in the AWS console and Terraform output during verification. The stack was destroyed after testing, so these identifiers are historical.

Resource	Mumbai (ap-south-1)	Singapore (ap-southeast-1)
VPC	vpc-0538546924b02ab6a	vpc-0383ca52ecb0800bf
VPC CIDR	10.1.0.0/16	10.2.0.0/16
Internet Gateway	igw-0a712dcf3bb891747	igw-0f25e16c53d7f3571
Route tables	rtb-070cc60e897a567da rtb-0f0af3ade38cdc36a	rtb-05c2739a767f3dcfe rtb-07134dd078899ee59
Availability Zones	ap-south-1a, ap-south-1b	ap-southeast-1a, ap-southeast-1b
Public subnets	10.1.0.0/24, 10.1.2.0/24	10.2.1.0/24, 10.2.2.0/24
Private subnets	10.1.22.0/24, 10.1.33.0/24	10.2.12.0/24, 10.2.13.0/24
EC2 instance	i-0b382a00186845521 10.1.22.250	i-0efdda59887197ac3 10.2.12.160
Load balancer	aws-disaster-recovery-v... (Active)	aws-disaster-recovery-v... (Active)
S3 bucket	parikshitarer-dr-primary-unique123	parikshitarer-dr-secondary-unique123

Appendix B: SSM Automation Document

Document name	Parikshit-disaster-recovery-dr-runbook
Action	Tag the Singapore EC2 instance
Tags applied	DRStatus = Failover-Ready DRActivationSource = Automated
Trigger	EventBridge rule matching CloudWatch alarm state change events
Not performed	Starting the instance, application validation, DNS update